

PHP and Laravel Cheat Sheet

Laravel 13.x (13.34.0) on PHP 8.3 to 8.5  
verified 2026-10-05 against a Laravel 13.34 application

**PLAIN PHP: THE WEB BASICS**

```
$_GET['q'] ?? ''; $_POST; $_SERVER['REQUEST_METHOD'];  
$_FILES; $_COOKIE;  
echo htmlspecialchars($_, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8'); // escape on output  
$st = $pdo->prepare('SELECT * FROM books WHERE id = :id');  
$st->execute(['id' => $id]); $book = $st->fetch(); // never concatenate SQL  
session_start(); session_regenerate_id(true); // at login  
header('Location: /books', true, 303); exit; //  
Post/Redirect/Get  
password_hash($pw, PASSWORD_DEFAULT); password_verify($pw, $hash);
```

**Sessions:** use `str_replace('cookie', 'httponly=1')`  
`cookie_secure=1` `cookie_samesite=Lax`. **Uploads:** check `['error']` and size, type from `info`, generated name outside the web root, `move_uploaded_file()`, re-encode images.  
**Serving:** Nginx → PHP-FPM over FastCGI, root `../public`, one front controller `index.php`, Opcache on.

**REQUEST LIFECYCLE**

`public/index.php` → `bootstrap/app.php` → kernel: load env and config, register then boot providers → global middleware → router matches the route → **web** group: cookies, session, `PreventRequestForgery`, bindings → route middleware (`auth`, `can`, `throttle`) → controller → response back through the middleware → sent; then terminable middleware and `defer()`. Under Octane the app boots once per worker.

**ARTISAN**

laravel new bookshelf; composer run dev // server, queue, logs and vite  
php artisan make:model Book -mfsf // migration, factory, seeder, controller  
php artisan make:controller BookController --resource --model=Book  
php artisan make:request StoreBookRequest; make:policy BookPolicy --model=Book  
php artisan make:job / make:event / make:listener / make:mail / make:notification  
php artisan make:component Alert; make:livewire BookSearch; make:test BookTest  
php artisan migrate; migrate:status; migrate:rollback; migrate:fresh -s seed  
php artisan queue:work --tries=3; queue:failed; queue:retry all  
php artisan install:api; install:broadcasting // Sanctum; Reverb and Echo  
php artisan route:list; about; tink; pail; schedule:list; model:show Book  
php artisan optimize; optimize:clear; reload; down; up

**BOOTSTRAP/APP.PHP**

```
return Application::configure(basePath: dirname(__DIR__))  
->withRouting(web: __DIR__.'/../routes/web.php',  
api: __DIR__.'/../routes/api.php',  
commands: __DIR__.'/../routes/console.php', health:  
'/up')  
->withMiddleware(function (Middleware $middleware): void  
{  
    $middleware->alias(['locale' => SetLocale::class]);  
    $middleware->trustProxies(at: ['10.0.0.0/8']);  
});  
->withExceptions(function (Exceptions $exceptions): void  
{  
    $exceptions->  
>dontReport([LockTimeoutException::class]);  
})->create();
```

Providers are listed in `bootstrap/providers.php`; there are no Kernel classes since Laravel 11.

**ROUTING**

```
Route::get('/books/{book}', [BookController::class, 'show'])->  
>name('books.show');  
Route::post('/books', [BookController::class, 'store'])->  
>middleware('auth');  
Route::get('/authors/{author:slug}', ...); // bind by column  
Route::get('/books/{id}', ...) ->whereNumber('id');  
Route::middleware(['auth', 'verified'])->prefix('admin')->  
>name('admin.')
```

`->group(function () { /* ... */ });`  
`Route::resource('books', BookController::class);` // `->only(...)`  
`Route::apiResource('books', ApiBookController::class);`  
`route('books.show', $book);`  
`URL::temporarySignedRoute('unsubscribe', now())->addDay()`, `...`  
`index GET /books show GET /books/{book}`  
`create GET /books/create edit GET /books/{book}/edit`  
`store POST /books update PUT/PATCH /books/{book}`  
`destroy DELETE /books/{book}`

**MIDDLEWARE**

```
public function handle(Request $request, Closure $next,  
string $role): Response  
{  
    abort_unless($request->user()?->hasRole($role), 403);  
    return $next($request); // or change the response after  
}  
Route::middleware('role:librarian'); // parameters after the colon  
#[Middleware('auth', except: ['index'])] // on a controller (Laravel 13)  
Aliases: auth guest verified can signed throttle password.confirm.
```

**CONTROLLERS, REQUESTS, RESPONSES**

```
public function store(StoreBookRequest $request, AddBook  
AddBook): RedirectResponse  
{  
    $book = $addBook->handle($request->validated());  
    return to_route('books.show', $book)->with('status',  
'Book added');  
}  
$request->input('title'); $request->integer('year');  
$request->boolean('ok');  
$request->query('page'); $request->file('cover'); $request->  
>user();  
return view('books.show', compact('book'));  
return response()->json($data, 201); return back()->  
>withErrors(...);  
return redirect()->route('books.index'); return response()->  
>download($path);  
abort(404); abort_if($cond, 403);
```

**VALIDATION**

```
$data = $request->validate([  
    'title' => ['required', 'string', 'max:200'],  
    'isbn' => ['required', 'Rule::unique('books')-  
>ignore($book)],  
    'author_id' => ['required', 'exists:authors,id'],  
    'year' => ['nullable', 'integer', 'between:1450,2026'],  
    'format' => ['required', 'Rule::enum(Format::class)],  
    'tags' => ['array', 'max:5'], 'tags.*' => ['string',  
'distinct'],  
    'cover' => ['nullable', 'file:image()->max('2mb')],  
    'email' => ['required', 'email'], 'starts_at' => ['date',  
'after:today'],  
]);  
class StoreBookRequest extends FormRequest {  
    public function authorize(): bool { return $this->user()-  
>can('create', Book::class); }  
    public function rules(): array { return [/* as above */];  
}
```

Failing: redirect back with `errors` and old input, or 422 JSON.  
`$request->safe()->only(...)`. `Password::defaults()` for passwords.

**BLADE**

```
{{ $book->title }} // escaped; {!! $html !!} only for trusted  
HTML  
@if ($books->isEmpty()) ... @elseif ... @else ... @endif  
@forelse ($books as $book) <li{{ $loop->iteration }}</li>  
@empty None @endforelse  
@auth ... @endauth @guest ... @endguest @can('update',  
$book) ... @endcan  
<form method="POST"> @csrf @method('PUT') <input name="title"  
value="{{ old('title') }}">  
@error('title') <p{{ $message }}</p> @enderror  
<x-layouts.app title="Books"> ... </x-layouts.app> // layout  
component  
<x-alert type="error" :message="$msg" /> // : passes PHP  
@props(['type' => 'info']) {{ $slot }} <x-slot:footer></x-  
slot:footer>  
<script>const book = { {{ Js::from($book) }};</script>  
@vite(['resources/js/app.js'])
```

**FRONT END: WHICH ONE**

**Blade + Vite** server-rendered pages, Alpine.js for small behaviour

**Livewire 4** interactive components in PHP: `wire:model`, `wire:click`, no API

**Inertia 3** React, Vue or Svelte pages fed by controllers: `Inertia::render('Books/Index', [...])`

**API + SPA** a separate front end: Sanctum cookies or tokens

Starter kits: React, Vue, Svelte or Livewire, with Fortify login, 2FA and passkeys.

**MIGRATIONS**

```
Schema::create('books', function (Blueprint $table) {  
    $table->id();  
    $table->foreignId('author_id')->constrained()-  
>cascadeOnDelete();  
    $table->string('title', 200); $table->string('isbn', 17)-  
>unique();  
    $table->unsignedSmallInteger('published_year')->  
>nullable();  
    $table->text('blurb')->nullable(); $table->json('meta')->  
>nullable();  
    $table->boolean('featured')->default(false); $table->  
>decimal('price', 8, 2);  
    $table->timestamps(); $table->softDeletes();  
    $table->index(['author_id', 'published_year']);  
});  
Schema::table('books', fn (Blueprint $t) => $t->  
>string('subtitle')->nullable()->after('title'));  
Also uuid() uuid() enum() timestampTz() renameColumn() dropColumn(). Never edit a migration that ran in production.
```

**ELOQUENT**

```
Book::query()->where('published_year', '>=', 2000)-  
>orderBy('title')->paginate(20);  
Book::findOrFail($id); Book::firstOrCreate(['isbn' => $isbn],  
[...]);  
Book::create($data); $book->update($data); $book->delete();  
Book::with('author')->withCount('reviews')->get(); // no N+1  
Book::whereHas('reviews', fn ($q) => $q->where('rating', 5))-  
>get();  
Book::where('stock', 0)->chunkById(500, fn ($books) => ...);  
public function author(): BelongsTo { return $this->  
>belongsTo(Author::class); }  
public function reviews(): HasMany { return $this->  
>hasMany(Review::class); }  
public function tags(): BelongsToMany { return $this->  
>belongsToMany(Tag::class); }  
protected function casts(): array {  
    return ['published_at' => 'datetime', 'format' =>  
Format::class, 'meta' => 'array'];  
}  
#[Fillable(['title', 'isbn'])] // Laravel 13; or fillable  
Model::shouldBeStrict(! app()->isProduction()); // lazy  
loading throws
```

**AUTH, POLICIES, SANCTUM**

```
if (Auth::attempt($credentials, remember: true)) { $request->  
>session(); }  
Auth::logout(); $request->session()->invalidate(); $request->  
>session()->regenerateToken();  
Gate::define('manage-loans', fn (User $u) => $u->  
>hasRole(Role::Librarian));  
public function update(User $user, Book $book): bool { return  
$user->id === $book->owner_id; }  
Gate::authorize('update', $book); $user->can('update',  
$book); // 403 or bool  
Route::put(...)->can('update', 'book'); #[Authorize('update',  
'book')]  
$token = $user->createToken('cli', ['books:write'])->  
>plainTextToken();  
Route::middleware('auth:sanctum')->get('/user', ...) $user->  
>tokenCan('books:write');
```

Sanctum: tokens and first-party SPA cookies. Passport only for a full OAuth 2 server.

**QUEUES, EVENTS, BROADCASTING, SCHEDULING**

```
#[Tries(3)] #[Backoff([1, 5])] #[Timeout(10)]  
class SyncBook implements ShouldQueue { use Queueable;  
    public function construct(public Book $book) {  
        public function handle(Catalogue $c): void { /*  
idempotent */ } }  
SyncBook::dispatch($book)->onQueue('catalogue')->delay(now()-  
>addMinutes(5));  
SyncBook::dispatch($book)->afterCommit(); // inside a  
transaction  
Bus::chain([new A, new B])->dispatch(); Bus::batch(...)-  
>dispatch();  
ReviewPosted::dispatch($review); // listeners discovered;  
ShouldQueue to queue  
class ReviewPosted implements ShouldBroadcast {  
    public function broadcastOn(): array { return [new  
PrivateChannel('books.', $this->review->book_id)]; } }  
Broadcast::channel('books.{id}', fn (User $u, int $id) =>  
true); // routes/channels.php  
Echo::private('books.1').listen('ReviewPosted', (e) => ...);  
// JavaScript  
Schedule::command('loans:remind-due')->dailyAt('08:00')->  
>onOneServer();  
* * * * * cd /srv/app && php artisan schedule:run // the one  
cron entry
```

**CACHE, MAIL, NOTIFICATIONS, STORAGE**

```
Cache::remember('popular', 600, fn () => Book::latest()-  
>take(10)->get());  
Cache::flexible('popular', [300, 900], fn () => ...); //  
serve stale, refresh  
Cache::forget('popular'); Cache::lock('import', 60)->get(fn  
() => ...);  
RateLimiter::for('api', fn (Request $r) =>  
Limit::perMinute(60)->by($r->user()?->id ? $r->ip());  
Mail::to($user)->queue(new ReviewModerated($review));  
$user->notify(new LoanDue($loan)); // via(): ['mail',  
'database']  
$path = $request->file('cover')->store('covers', 's3');  
Storage::disk('s3')->temporaryUrl($path, now()->  
>addMinutes(5));  
Storage::disk('local')->get($path); // private by default  
php artisan storage:link for the public disk.  
__('books.added') and lang/es.json for translations.
```

**TESTING WITH PEST**

```
uses(RefreshDatabase::class);  
it('stores a book', function () {  
    Queue::fake();  
    $this->actingAs(User::factory()->create(['role' =>  
Role::Librarian]));  
    ->post('/books', Book::factory()->raw())  
    ->assertRedirect()->assertSessionHasNoErrors();  
    Queue::assertPushed(SyncBook::class);  
});  
$this->postJson('/api/v1/books', [])->assertUnprocessable()-  
>assertInvalid();  
$this->getJson('/api/v1/books/1')->assertOk()-  
>assertJsonPath('data.id', 1);  
Mail::fake(); Notification::fake(); Event::fake();  
Storage::fake('s3');  
Http::fake(['catalogue.test/*' => Http::response('Ok' =>  
true)]); Http::preventStrayRequests();  
php artisan test --parallel --coverage // or vendor/bin/pest
```

**DEPLOYMENT CHECKLIST**

```
composer install --no-dev --optimize-autoloader . npm ci &&  
npm run build . php artisan migrate --force . php artisan  
optimize . APP_ENV=production APP_DEBUG=false env() only in  
config/ . Opcache with validate_timestamps=0, reload FPM .  
php artisan reload (workers, Octane, Reverb) . workers under  
Supervisor or Horizon . cron schedule:run . storage:link .  
trusted proxies and hosts . /up health check . logs to stderr .  
composer audit . one image for web, worker and scheduler.
```

**CHANGED SINCE OLDER TUTORIALS**    `app/User.php` → `app/Models` (8)    `factory(User::class)` → class factories (8)    `HTTP/Console Kernel` → `bootstrap/app.php` (11)    **Mix** → Vite (9.19)  
`$dates` → `casts()` (removed 10)    `make:auth` → starter kits, Fortify (6/12)    `VerifyCsrfToken` → `PreventRequestForgery` (13)    **Homestead/Valet** → Herd, Sail, Docker  
`str_slug()`, `array_get()` → `Str::slug()`, `Arr::get()` (6)    `Controller@method` → `[Controller::class, 'method']` (8)